



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παραλληλα Συστήματα
Εργασία 3

Ονοματεπώνυμο: ΠΑΚΙΤΣΑΚ ΝΤΕΛΙΑ-ΜΑΡΙΑ /ΙΑΣΩΝ ΚΑΡΑΠΡΟΔΡΟΜΙΔΗΣ
Αριθμός Μητρώου: 1115202200125 / 1115202200064

Περιεχόμενα

Εισαγωγή	2
1 Άσκηση 1	2
1.1 Περιγραφή Προβλήματος	2
1.2 Περιγραφή λύσης	2
1.3 Πειραματικά αποτελέσματα και σχολιασμός	3
2 Άσκηση 2	7
2.1 Περιγραφή Προβλήματος	7
2.2 Περιγραφή Λύσης	7
2.3 Πειραματικά αποτελέσματα και σχολιασμός	7
3 Άσκηση 3	16
3.1 Περιγραφή Προβλήματος και λύσης	16
3.2 Πειραματικά αποτελέσματα και σχολιασμός	16

Εισαγωγή

Για την παραγωγή γραφημάτων χρησιμοποιήσαμε *script* σε *python*. Όλες οι ασκήσεις εκτελέστηκαν στα μηχανήματα *linux* της σχολής τα οποία διαθέτουν 4 πυρήνες με 1 νήμα ανά πυρήνα. Οι χρόνοι στα πειράματα είναι ο μέσος όρος από 4 εκτελέσεις. Χρησιμοποιήσαμε *Makefile* για το *build*.

1 Ασκήση 1

1.1 Περιγραφή Προβλήματος

Η ασκήση ζητά να υλοποιήσουμε ένα πρόγραμμα που υπολογίζει τον πολλαπλασιασμό δυο τυχαίων πλήρων πολυωνυμικών βαθμού n με μη μηδενικούς ακέραιους συντελεστές χρησιμοποιώντας τον παράλληλο αλγόριθμο *MPI*. Το πρόβλημα δομείται γύρω από τον κεντρικό ρόλο της διεργασίας 0 η οποία λειτουργεί ως συντονιστής. Πιο συγκεκριμένα είναι υπεύθυνη για τη δημιουργία των τυχαίων πολυωνύμων, τη διανομή των απαραίτητων δεδομένων στις υπολοίπες διεργασίες και την τελική συγκεντρώνση των επιμέρους τμημάτων για τον σχηματισμό του τελικού πολυωνύμου. Στην συνέχεια θα πρέπει να μετρήσουμε και να συγκρίνουμε τους χρόνους αποστολής δεδομένων, του παράλληλου υπολογισμού του χρόνου λήψης δεδομένων καθώς και τον συνολικό χρόνο. Τέλος θα πρέπει να συγκρίνουμε τα αποτελέσματα του παράλληλου υπολογισμού με *MPI* έναντι του παράλληλου υπολογισμού με *Pthreads* (εργασία 1) και με *OpenMP* (εργασία 2).

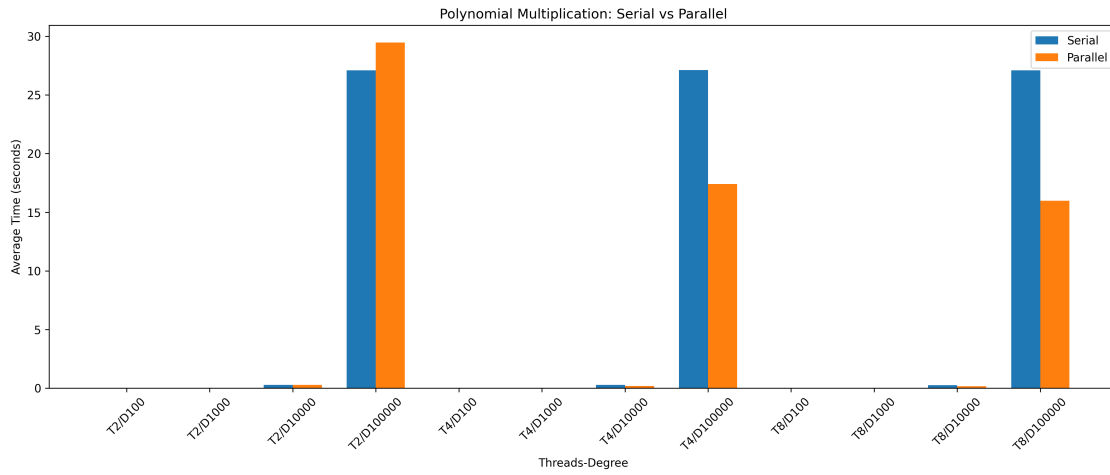
1.2 Περιγραφή λύσης

Αρχικά το πρόγραμμα ελέγχει αν τα ορίσματα που δέχεται είναι σωστά, δηλαδή αν δοθηκε τιμή για τον βαθμό των πολυωνύμων. Η διεργασία 0 ($rank = 0$) αναλαμβάνει την δημιουργία των πινάκων που θα περιέχουν τους τυχαίους μη μηδενικούς συντελεστές με την χρήση της συνάρτησης *generate_polynomials*.

Για την υλοποίηση του παράλληλου υπολογισμού χρησιμοποιούμε τη στρατηγική του διαχωρισμού δεδομένων. Η διεργασία 0 (*master*) στέλνει ολόκληρο το δεύτερο πολυώνυμο (P_2) σε όλες τις υπολοίπες διεργασίες με την χρήση την εντολής *MPI_Bcast* διότι για τον πολλαπλασιασμό πολυωνύμων κάθε συντελεστής a_i του πρώτου πολυωνύμου P_1 πρέπει να πολλαπλασιαστεί με όλους τους συντελεστές b_j του δεύτερου πολυωνύμου. Αντιθέτως το πρώτο πολυώνυμο (P_1) μοιράζεται σε τμήματα (*chunks*) στις διεργασίες χρησιμοποιώντας την *MPI_Scatterv*. Η χρήση της *Scatterv* αντί για την απλή *Scatter* είναι απαραίτητη διότι υπάρχει περίπτωση ο αριθμός των συντελεστών ($n + 1$) να μην διαιρείται ακριβώς με τον αριθμό των διεργασιών. Έτσι φροντίζουμε το υπόλοιπο της διαιρέσης να κατανεμεται δίκαια και να μην χάνονται δεδομένα. Ταυτόχρονα όσο θα γίνεται αυτή η διαδικασία θα μετρήσουμε τον χρόνο αποστολής δεδομένων με την χρήση *MPI_Wtime()*

Για το στάδιο του υπολογισμού με παράλληλο υπολογισμό κάθε διεργασία διαθέτει ένα τοπικό πίνακα του P_1 (*temp_P1*) και ολόκληρο το P_2 . Κάθε διεργασία εκτελεί έναν τοπικό πολλαπλασιασμό όπου κάθε στοιχείο του πίνακα (*temp_P1*) πολλαπλασιάζεται με όλα τα στοιχεία του P_2 . Έτσι το αποτέλεσμα θα το αποθηκεύσουμε σε έναν τοπικό πίνακα *local_result* μεγέθους $2n + 1$. Με τον υπολογισμό του κατάλληλου *offset* (*start_i*) διασφαλίζουμε ότι κάθε διεργασία τοποθετεί τα γινόμενα στις σωστές θέσεις που αντιστοιχούν στους βαθμούς του τελικού πολυωνύμου. Στην συνέχεια η διεργασία 0 συγκεντρώνει τα αποτελέσματα από τους τοπικούς πίνακες *local_result*. Πιο συγκεκριμένα κάθε διεργασία εκτός από την διεργασία 0 στέλνει τον πίνακα *local_result* μέσω της *MPI_Send* και η διεργασία 0 τα λαμβάνει με την *MPI_Recv*. Στη συνέχεια ο *master* αθροίζει αυτά τα αποτελέσματα (μαζί με το δικό του αποτέλεσμα που υπολόγισε) στον τελικό πίνακα *final_result*. Τέλος για να αξιολογήσουμε την απόδοση μετράμε και τυπώνουμε τους χρόνους εκτέλεσης του παράλληλου υπολογισμού και λήψης των αποτελεσμάτων των διεργασιών από τη διεργασία 0

1.3 Πειραματικά αποτελέσματα και σχολιασμός



(α') Times for the parallel multiplication using Pthreads (1.1 exercise)

Summary of Average Times and Speedup

Threads-Degree	Serial	Parallel	Improvement (Serial/Parallel)
T2/D100	0.000028	0.000093	0.30x
T2/D1000	0.002890	0.001450	1.99x
T2/D10000	0.274216	0.131344	2.09x
T2/D100000	27.050443	13.174836	2.05x
T4/D100	0.000028	0.000130	0.22x
T4/D1000	0.002746	0.001290	2.13x
T4/D10000	0.272349	0.100753	2.70x
T4/D100000	27.154960	10.043762	2.70x
T8/D100	0.000028	0.000333	0.08x
T8/D1000	0.002839	0.001371	2.07x
T8/D10000	0.268679	0.086653	3.10x
T8/D100000	27.022040	7.922539	3.41x

(β') Times for the parallel multiplication using OpenMP (1.2 exercise)

Serial Timing Only

n	Processes	Serial (s)	Total (s)	Improvement (serial/total)
100	2	0.000046	0.000073	0.63x
100	4	0.000114	0.000154	0.74x
100	8	0.000129	0.002668	0.05x
1000	2	0.003300	0.001755	1.88x
1000	4	0.003316	0.000930	3.57x
1000	8	0.010777	0.003793	2.84x
10000	2	0.292735	0.141689	2.07x
10000	4	0.297933	0.075524	3.94x
10000	8	0.292690	0.041696	7.02x
100000	2	27.912822	14.248935	1.96x
100000	4	29.216456	7.516001	3.89x
100000	8	29.220042	3.802702	7.68x

(γ') Times for the parallel multiplication using MPI (1.3 exercise)

Σχήμα 1: Comparison of parallel multiplication methods.

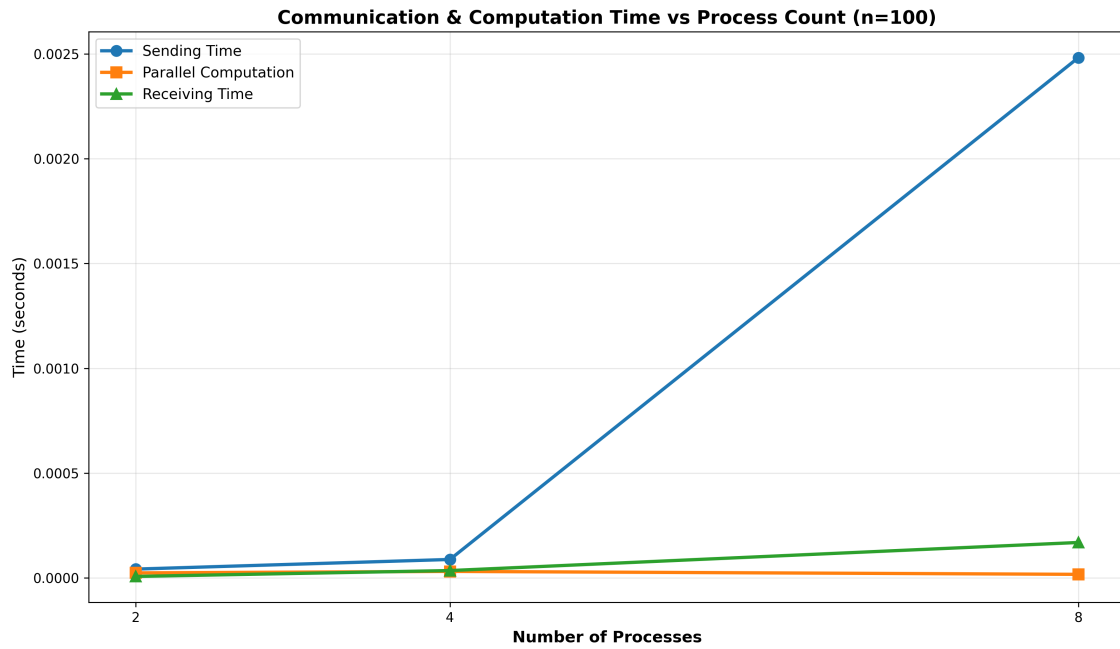
Όπως παρατηρούμε από τα πειραματικά αποτελέσματα που συλλεχθήκαν από το *script.py* η απόδοση του παραλλήλου αλγορίθμου εξαρτάται έντονα από τον βαθμό του πολυωνυμίου και τον αριθμό των *threads*. Με βάση τα παραπάνω γραφήματα της άσκησης 1.1 άσκηση 2.1 και άσκηση 3.1 παρατηρούμε ότι για μικρά μεγέθη εισόδου ($n = 100$) η παραλληλοποίηση αποτυγχάνει να προσφέρει βελτίωση ($speedup < 1x$) και στις μεθόδους (*Pthreads*, *OpenMP* και *MPI*) καθώς το *OVERHEAD* της δημιουργίας διεργασιών ή νημάτων και ο συντονισμός τους από τον *SCHEDULER* του λειτουργικού συστήματος υπερκαλύπτουν το υπολογιστικό κέρδος. Στην περίπτωση του *MPI* (Εικόνα 3) η χρήση 8 διεργασιών για $n = 100.000$ επιτυγχάνει την κορυφαία επίδοση ($7.68x$) γεγονός που εξηγείται από τη φύση της αρχιτεκτονικής *DISTRIBUTED MEMORY*. Το πρόγραμμα *MPI* εκτελείται σε 5 παραλλήλους υπολογιστές *Linux* του πανεπιστημιακού δικτύου αξιοποιώντας το άθροισμα του *MEMORY BANDWIDTH* και της *L3 CACHE* πολλαπλών κομβίων. Η πλήρης απομονώση του *ADDRESS SPACE* κάθε διεργασίας επιτρέπει στη *CPU* να διαχειρίζεται τα δεδομένα χωρίς το κόστος του *CACHE COHERENCY PROTOCOL* το οποίο επιβαρύνει σημαντικά το *OpenMP* (Εικόνα 2, $3.41x$) και τα *Pthreads* (Εικόνα 1, $1.69x$). Πιο συγκεκριμένα στα *Pthreads* και *OpenMP* επειδή όλοι οι επεξεργαστές μοιράζονται την ίδια μνήμη στον ίδιο υπολογιστή όταν προσπαθούν να διαβάσουν ή να γράψουν ταυτόχρονα στα ίδια δεδομένα ο ένας πρέπει να περιμένει τον άλλον. Επίσης αν ένας επεξεργαστής αλλάξει κάτι η *CPU* πρέπει να ενημερώσει όλους τους άλλους επεξεργαστές κάτι που προκαλεί μεγάλες καθυστερήσεις. Αντιθέτα στο *MPI* επειδή το πρόγραμμα τρέχει σε 5 διαφορετικούς υπολογιστές είναι σαν να έχουμε 5 ανεξάρτητους δρόμους. Κάθε υπολογιστής έχει τη δική του μνήμη και δεν χρειάζεται να περιμένει κανέναν άλλον. Παρόλο που οι υπολογιστές πρέπει να μιλάνε μεταξύ τους μέσω δικτύου (που είναι πιο αργό από τη μνήμη) όταν τα πολυωνύμα είναι πολύ μεγάλα ($n = 100.000$) ο χρόνος που κερδίζουν είναι πολύ μεγαλύτερος από τον χρόνο που χάνουν για να στείλουν μηνύματα μέσω του δικτύου.

Timing (Total = Sending + Parallel + Receiving)

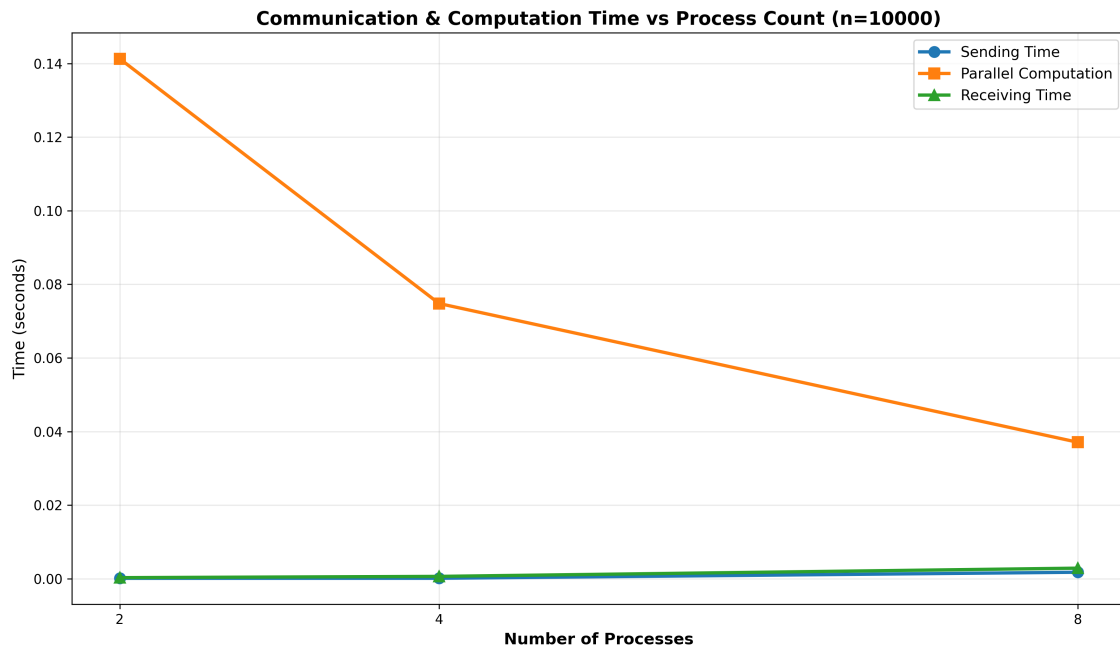
n	Processes	Total (s)	Sending (s)	Parallel (s)	Receiving (s)
100	2	0.000073	0.000042	0.000024	0.000007
100	4	0.000154	0.000088	0.000031	0.000035
100	8	0.002668	0.002482	0.000017	0.000169
1000	2	0.001755	0.000043	0.001689	0.000023
1000	4	0.000930	0.000039	0.000846	0.000045
1000	8	0.003793	0.002082	0.001367	0.000344
10000	2	0.141689	0.000101	0.141304	0.000284
10000	4	0.075524	0.000110	0.074787	0.000627
10000	8	0.041696	0.001753	0.037079	0.002864
100000	2	14.248935	0.000431	14.246419	0.002085
100000	4	7.516001	0.000863	7.511423	0.003715
100000	8	3.802702	0.006880	3.743160	0.052662

Σχήμα 2: The sending , receiving and parallel times of the multiplication using MPI

Επιπλέον εξετάζοντας ειδικότερα τη δομή του χρόνου στο MPI από τον παραπάνω πίνακα βλέπουμε ότι για μεγάλα πολυωνύμα ο χρόνος εκτέλεσης του παράλληλου υπολογισμού (*Parallel*) κυριαρχεί φτάνοντας τα $3.743160s$ για 8 διεργασίες ενώ οι χρόνοι *Sending* και *Receiving* παραμένουν εξαιρετικά χαμηλοί ($0.006880s$ και $0.052662s$ αντιστοίχα). Αυτό αποδεικνύει ότι το δίκτυο των 5 υπολογιστών δεν αποτελεί συμφορηση για το συγκεκριμένο πρόβλημα. Η αποδοτικότητα παραμένει υψηλή στις μεγάλες τιμές του n γιατί το κόστος αποστολής και λήψης (*Sending + Receiving*) των δεδομένων από τη διεργασία 0 είναι αμελητέο μπροστά στο όφελος του παραλλήλου υπολογισμού. Αντιθέτα για χαμηλά n ($n = 100$) η σχέση αυτή αντιστρέφεται δραματικά. Για παράδειγμα με 8 διεργασίες ο χρόνος *Sending* ($0.002482s$) είναι υπερπολλαπλασιασμός του καθαρού χρόνου υπολογισμού *Parallel* ($0.000017s$).



(α') Impact of process count on communication and computation times for a fixed polynomial degree 100



(β') Impact of process count on communication and computation times for a fixed polynomial degree 10000

Σχήμα 3: Graphs with fixed degree number

Εχουμε επισυναψει 2 επιπλεον γραφηματα που βλεπουμε την συμπεριφορα καθως αυξανονται οι αριθμοι των *processes* με σταθερο βαθμο πολυωνθμου n . Η συμπεριφορα των χρονων διαφοροποιουνται αναλογα με το μεγαθος του προβληματος.Εδω βλεπουμε και σχηματικα τις ιδιες παρατηρησεις που καναμε προηγουμενως για τους χρονους (*Sending,Receiving , Parallel*). Επιπλεον για $n=100.000$ η αυξηση των διεργασιων οδηγει σε βελτιωση της αποδοσης. Καθως ανεβαινουμε απο τις 2 στις 8 διεργασίες ο καθαρος χρονος υπολογισμου (*Parallel*) υποδιπλασιαζεται πεφτοντας απο τα 0.141304s στα 0.037079s. Σε αυτο το επιπεδο ο ογκος των υπολογισμων ειναι τοσο μεγαλος που ‘‘κρυβει’’ τις καθυστερησεις του δικτυου.

Timing (Total = Sending + Parallel + Receiving)

n	Processes	Total (s)	Sending (s)	Parallel (s)	Receiving (s)
100	2	0.000075	0.000039	0.000030	0.000006
100	4	0.029761	0.000039	0.000014	0.029708
100	8	0.045412	0.026265	0.000008	0.019139
1000	2	0.003119	0.000086	0.003007	0.000026
1000	4	0.010312	0.000048	0.001340	0.008924
1000	8	0.087792	0.078309	0.000664	0.008819
10000	2	0.405055	0.000098	0.404598	0.000359
10000	4	0.260111	0.000144	0.255824	0.004143
10000	8	0.148607	0.007083	0.111018	0.030506
100000	2	33.002495	0.000465	32.999503	0.002527
100000	4	19.777919	0.017876	19.739554	0.020489
100000	8	11.632364	0.022995	10.529772	1.079597

(α') Detailed timing breakdown for MPI parallel multiplication using 10 machines showing Sending, Parallel Computation, and Receiving components.

Serial Timing Only

n	Processes	Serial (s)	Total (s)	Improvement (serial/total)
100	2	0.000053	0.000075	0.71x
100	4	0.000050	0.029761	0.00x
100	8	0.000052	0.045412	0.00x
1000	2	0.005526	0.003119	1.77x
1000	4	0.004802	0.010312	0.47x
1000	8	0.004727	0.087792	0.05x
10000	2	0.569560	0.405055	1.41x
10000	4	0.982272	0.260111	3.78x
10000	8	0.709354	0.148607	4.77x
100000	2	62.582104	33.002495	1.90x
100000	4	68.979341	19.777919	3.49x
100000	8	73.688190	11.632364	6.33x

(β') Serial vs. Total Parallel execution times and speedup (Improvement) for 10 distributed Linux machines

Σχήμα 4: Tables with times for 10 distributed Linux workstations

Τέλος εκτελέσαμε το πρόγραμμα ξανά για 10 παραλληλούς υπολογιστές αντι για 5 που τρεχαμε μέχρι τώρα. Από τους παραπάνω πίνακες παρατηρούμε μείωση της βελτίωσης από 7.68x (στους 5 υπολογιστές) σε 6.33x (στους 10 υπολογιστές) για $n = 100.000$. Παρόλο που θεωρητικά η προσθήκη περισσότερων υπολογιστών θα έπρεπε να προσφέρει μεγαλύτερη επεξεργαστική ισχύ στην πράξη η απόδοση επηρεάζεται από την ανάγκη συντονισμού περισσότερων ανεξάρτητων κομβών μέσω του δικτύου. Στην περίπτωση των 10 υπολογιστών ο χρόνος *Sending* και *Receiving* αυξάνεται σημαντικά. Συγκεκριμένα η διεργασία 0 πρέπει πλέον να επικοινωνήσει με διπλάσιο αριθμό μηχανημάτων κάτι που αυξάνει τις καθυστερήσεις στην αναμονή των πακετών δεδομένων. Ενώ στους 5 υπολογιστές ο χρόνος επικοινωνίας ήταν αμελητέος μπροστά στον υπολογισμό στους 10 υπολογιστές το κόστος του διαμοιρασμού των δεδομένων αρχίζει να αφαιρεί από το κέρδος της παραλληλοποίησης.

2 Ασκήση 2

2.1 Περιγραφή Προβλήματος

Έπρεπε να υλοποιήσουμε ένα πρόγραμμα που πραγματοποιεί πολλαπλασιασμό πίνακα με διάνυσμα σε δύο διαφορετικές μορφές: μία με πυκνό πίνακα, και μία με τη μορφή CSR.

Η μορφή αυτή χωρίζει τον Matrix σε τρεις πίνακες: 1. Έναν με τα μη μηδενικά στοιχεία. 2. Έναν με το index της στήλης στην οποία βρίσκεται το κάθε στοιχείο. 3. Έναν που κρατάει τον αριθμό των μη μηδενικών στοιχείων πριν από κάθε σειρά.

Με αυτόν τον τρόπο, μπορούμε να εκτελούμε τον πολλαπλασιασμό πολύ πιο γρήγορα σε αραιούς πίνακες, καθώς αποφεύγουμε την επεξεργασία των μηδενικών στοιχείων. Στη συνέχεια, έπρεπε να παραλληλοποιήσουμε το πρόγραμμα με τη χρήση του MPI και να συγκρίνουμε τα αποτελέσματα για διάφορες παραμέτρους.

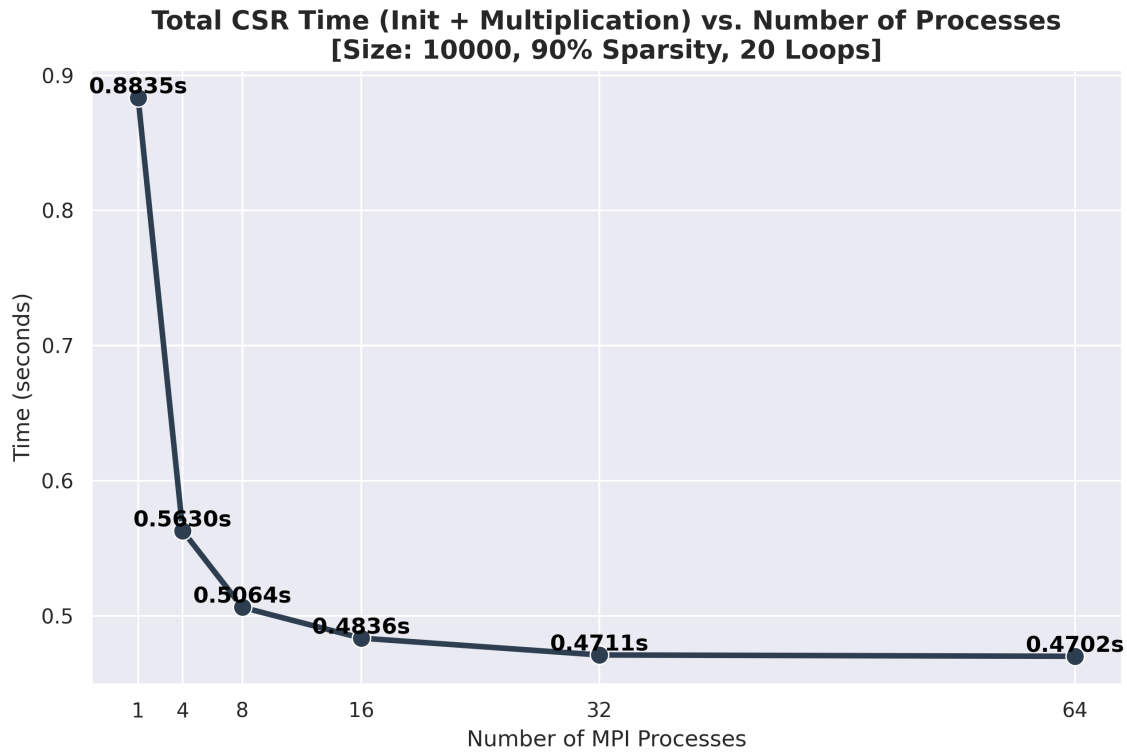
2.2 Περιγραφή Λύσης

Για τους πίνακες χρησιμοποίησα μια μονοδιάστατη αναπαράσταση. Με αυτόν τον τρόπο μου ήταν πιο εύκολο να μετατρέψω τον πίνακα στη μορφή που θέλαμε. Για την πυκνή αναπαράσταση, τροποποίησα τον κώδικα του βιβλίου ώστε να δουλεύει με τα δεδομένα του προγράμματος. Για την CSR αναπαράσταση, πρώτα μετέτρεψα τα δεδομένα στη μορφή των 3 πινάκων και μετά υλοποίησα τον αλγόριθμο για τον πολλαπλασιασμό.

Για να παραλληλοποιήσω τον κώδικα με MPI, έπρεπε να βρω έναν αποδοτικό τρόπο να μοιράσω τα δεδομένα στις διεργασίες. Για την CSR αναπαράσταση επέλεξα να μοιράσω τα δεδομένα κατά γραμμή, αρχικά βρίσκοντας ποιες γραμμές θα έπαιρνε η κάθε διεργασία και μετά μοιράζοντας τόσο τους πίνακες με τα δεδομένα όσο και τις πληροφορίες για τις στήλες. Επειδή όμως στην κατασκευή του πίνακα `rowIdx` κάθε τιμή του σηματοδοτεί πού τελειώνει μια γραμμή αλλά και πού αρχίζει η επόμενη, κάθε διεργασία χρειαζόταν οι επιμέρους πίνακες του `rowIdx` να είναι επικαλυπτόμενοι. Για αυτόν τον λόγο χρησιμοποίησα την `MPI_Scatterv` αντί της κανονικής, ώστε να μπορώ να ελέγξω καλύτερα ποια δεδομένα θα σταλθούν και πού. Επίσης, χρησιμοποίησα την `MPI_Scatterv` και για τα υπόλοιπα δεδομένα της CSR αναπαράστασης, διότι κάθε διεργασία θα λάμβανε διαφορετικό αριθμό δεδομένων. Τα διανύσματα τα έκανα Broadcast, διότι κάθε διεργασία έπρεπε να έχει στη διάθεσή της ολόκληρο το διάνυσμα. Τέλος, για τη συλλογή των δεδομένων χρησιμοποίησα την `MPI_Allgather`, ώστε όλες οι διεργασίες να έχουν το ορθό διάνυσμα για να γίνουν οι υπολογισμοί σωστά. Η `variable` έκδοση της συνάρτησης χρησιμοποιήθηκε για τον ίδιο λόγο που χρησιμοποιήθηκε και στο μοίρασμα των πινάκων `V` και `colIdx`. Χρησιμοποίησα την `MPI_Barrier` γύρω από όλες τις μετρήσεις χρόνου, ώστε να μπορώ να έχω όσο το δυνατόν πιο ορθά αποτελέσματα. Προφανώς, σε ένα πραγματικό σενάριο δεν χρειάζονται βρίσκονται εκεί αποκλειστικά και μόνο για την ορθή μέτρηση του χρόνου εκτέλεσης, καθώς το πρόγραμμα τρέχει σωστά και χωρίς τα `barriers`.

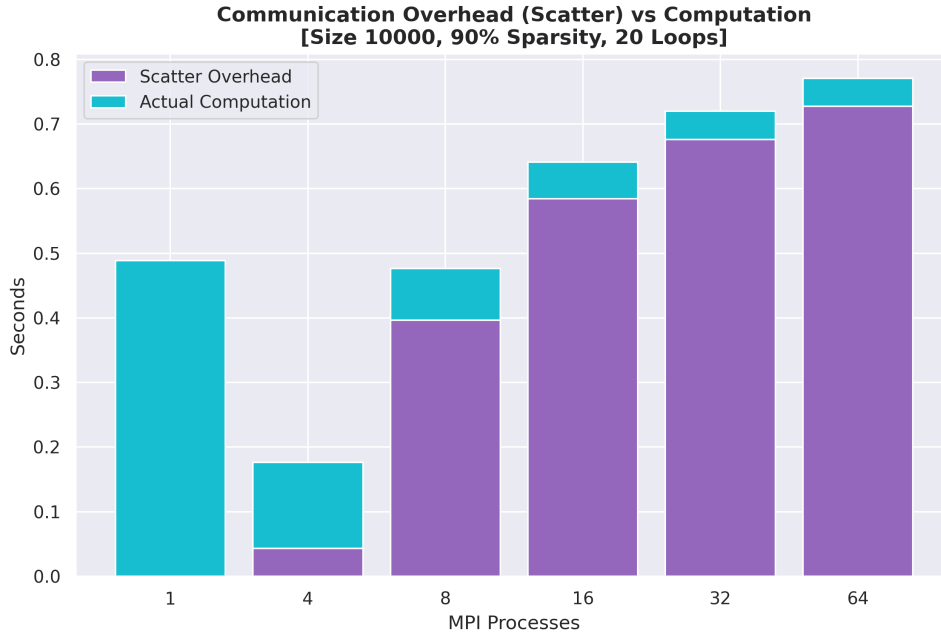
2.3 Πειραματικά αποτελέσματα και σχολιασμός

Για αρχή έκανα πειράματα για να δω πόσο καλά παραλληλοποιείται η CSR αναπαράσταση του προγράμματος. Στη συνέχεια παρατίθενται τα γραφήματα που προέκυψαν από την εκτέλεση των πειραμάτων:

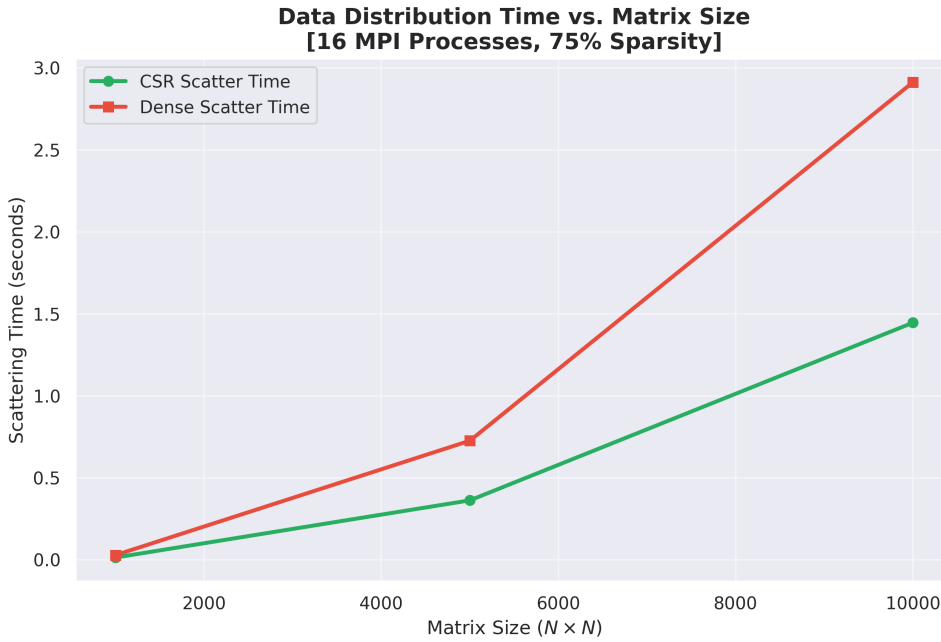


Σχήμα 5: CSR multiplication scaling with variable number of processes.

Όπως φαίνεται, το πρόγραμμα παρουσιάζει αρκετά καλό scaling με περισσότερες διεργασίες, καθώς υπάρχει αρκετός όγκος εργασίας που μπορεί να επιτελέσει ο κάθε κόμβος, οπότε σημειώνεται αισθητή βελτίωση. Παρόλα αυτά, υπάρχει ένα μεγάλο overhead, το οποίο προκύπτει από την ανάγκη επικοινωνίας. Τα πειράματα πραγματοποιήθηκαν στους υπολογιστές της σχολής και ο χρόνος που απαιτούνταν για τη μεταφορά των δεδομένων ήταν μεγαλύτερος από τον χρόνο του πραγματικού υπολογισμού:



(α') Decomposition of execution time into MPI scattering overhead and computational runtime across varying process counts.



(β') Scaling characteristics of data distribution latency as a function of matrix dimension N .

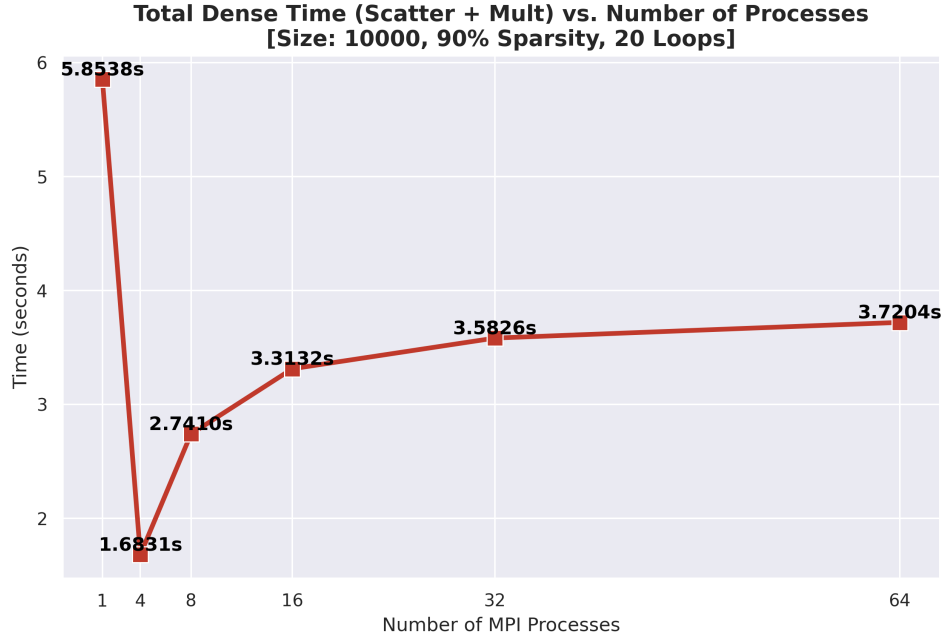
Σχήμα 6: Characterization of communication overhead in parallel matrix operations: (a) illustrates the relative cost of data distribution compared to computation, while (b) demonstrates the impact of problem scale on total scattering time.

Παρατηρούμε εδώ ότι, όταν χρησιμοποιείται παραπάνω από ένα μηχάνημα (από 8 διεργασίες και πάνω, διότι κάθε μηχάνημα διαθέτει 4 πυρήνες), ο συνολικός χρόνος εκτέλεσης ολοένα και αυξάνεται, φτάνοντας να

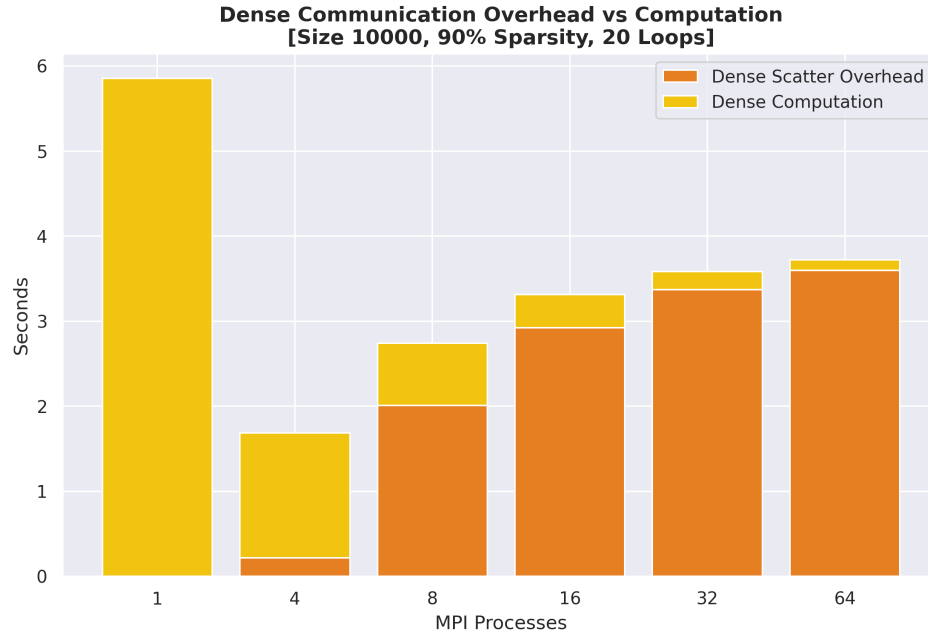
υπερβαίνει αυτόν του σειριακού προγράμματος, παρόλο που ο πραγματικός χρόνος του υπολογισμού συνεχώς μειώνεται.

Όπως φαίνεται, όσο περισσότερους κόμβους και δεδομένα πρέπει να επεξεργαστεί το πρόγραμμα, τόσο μεγαλύτερο overhead παρουσιάζει η επικοινωνία μεταξύ των κόμβων. Αυτό αποτελεί ένα χαρακτηριστικό παράδειγμα που δείχνει την ανάγκη για ταχύτατη μεταφορά δεδομένων ανάμεσα στους κόμβους, ειδικά όταν υπάρχει μεγάλος όγκος δεδομένων, όπως συμβαίνει στην πραγματικότητα σε ένα σύστημα HPC. Όσο κι αν προσπαθήσαμε να περιορίσουμε αυτό το overhead, αυτό παρέμενε μεγαλύτερο από τον χρόνο των υπολογισμών.

Αυτό το πρόβλημα είναι ακόμα πιο εμφανές στη dense αναπαράσταση, η οποία, παρόλο που επίσης δείχνει να έχει καλό scaling με περισσότερες διεργασίες, λόγω του ότι πρέπει να μεταφέρει μεγαλύτερο όγκο δεδομένων, ο συνολικός χρόνος εκτέλεσης είναι πολύ μεγαλύτερος:



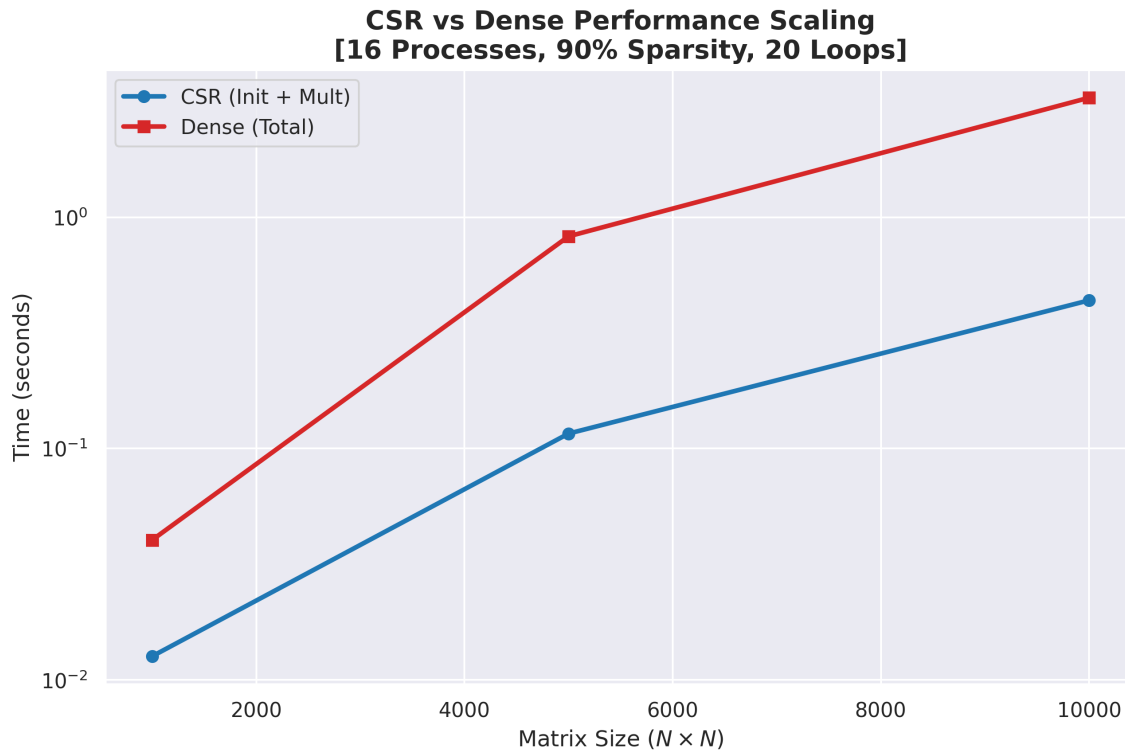
(α') Total execution time (incorporating data distribution and computation) as a function of MPI process count.



(β') Compositional analysis of execution time, distinguishing between communication overhead (Scatter) and computational workload.

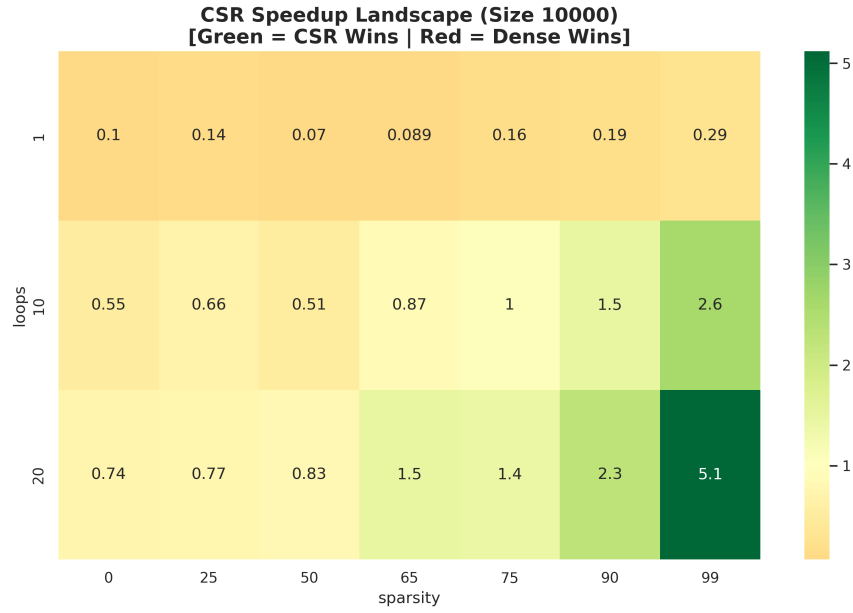
Σχήμα 7: Performance and scalability analysis for Dense matrix multiplication: (a) demonstrates the impact of parallelization on the overall execution time, while (b) illustrates the increasing dominance of communication overhead as the number of processes grows.

Όσον αφορά τον πραγματικό υπολογισμό, και οι δύο αναπαραστάσεις φαίνεται να έχουν αρκετά παρόμοια αύξηση στον χρόνο εκτέλεσης όσο αυξάνεται το μέγεθος του πίνακα.

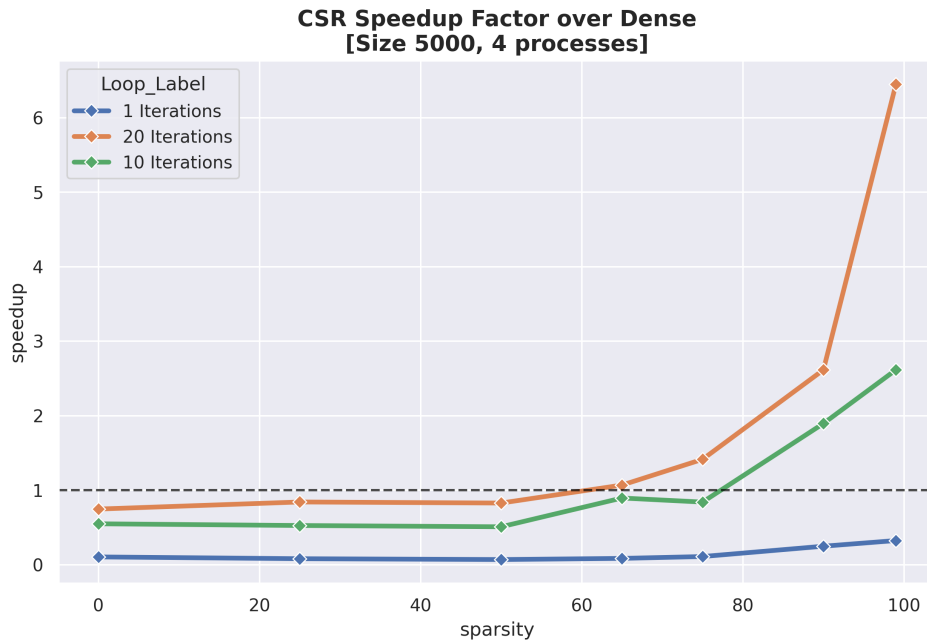


Σχήμα 8: Time graph of dense and CSR while matrix size grows.

Σε σύγκριση με την πυκνή μορφή, η αναπαράσταση CSR, μετά από πειράματα, φαίνεται να είναι πιο γρήγορη όταν οι πίνακες έχουν περισσότερα μηδενικά στοιχεία και όταν κάνουμε παραπάνω επαναλήψεις. Όσο πιο αραιοί είναι οι πίνακες, τόσο μεγαλύτερο speedup έχει η αναπαράσταση CSR, όπως δείχνει το παρακάτω heatmap:



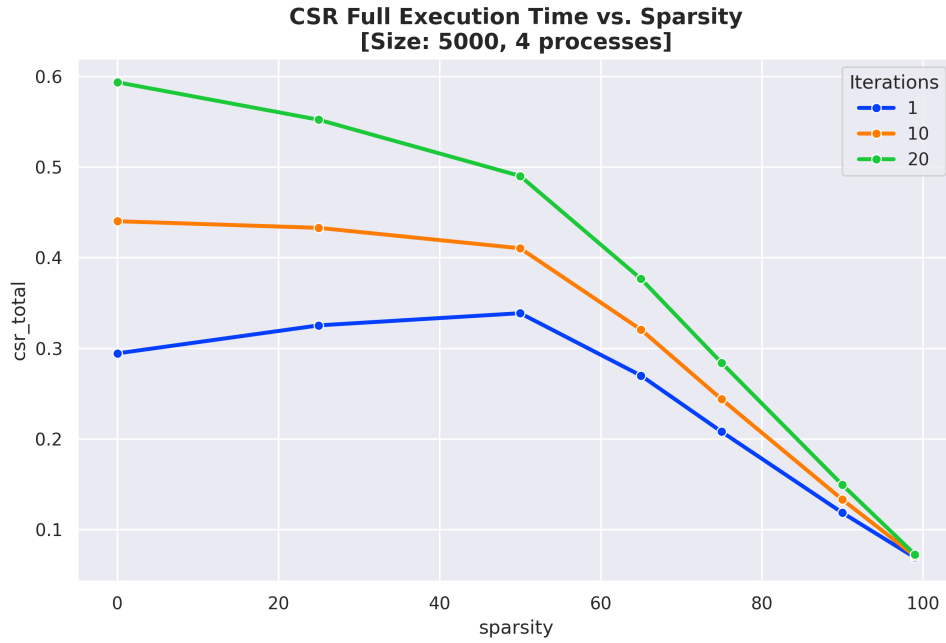
(α') Heatmap analysis showing the efficiency landscape of CSR across various iteration counts and sparsity levels.



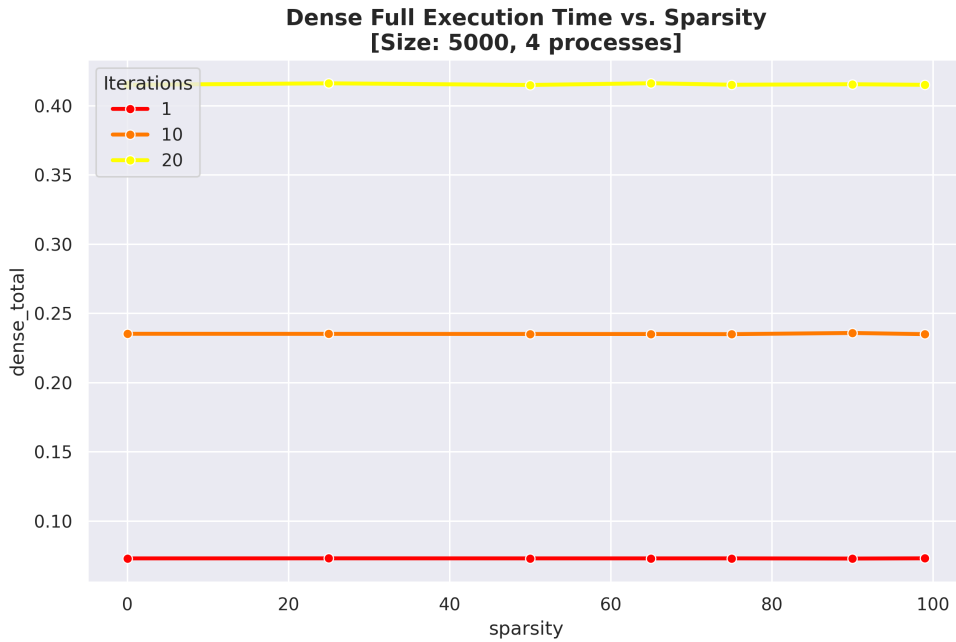
(β') Measured speedup ratio of the CSR implementation relative to the dense baseline as a function of sparsity.

Σχήμα 9: Comprehensive efficiency analysis: (a) provides a holistic view of performance gains (green regions) and losses (red regions), while (b) quantifies the acceleration factor achieved as the matrix becomes increasingly sparse.

Όπως δείχνει αυτό το heatmap, η dense αναπαράσταση, όπως και θα περιμέναμε, δεν κάνει καλά scale με περισσότερα μηδενικά στοιχεία. Όπως δείχνουν επίσης και τα παρακάτω γραφήματα:



(α') Total execution time for CSR format across sparsity levels.



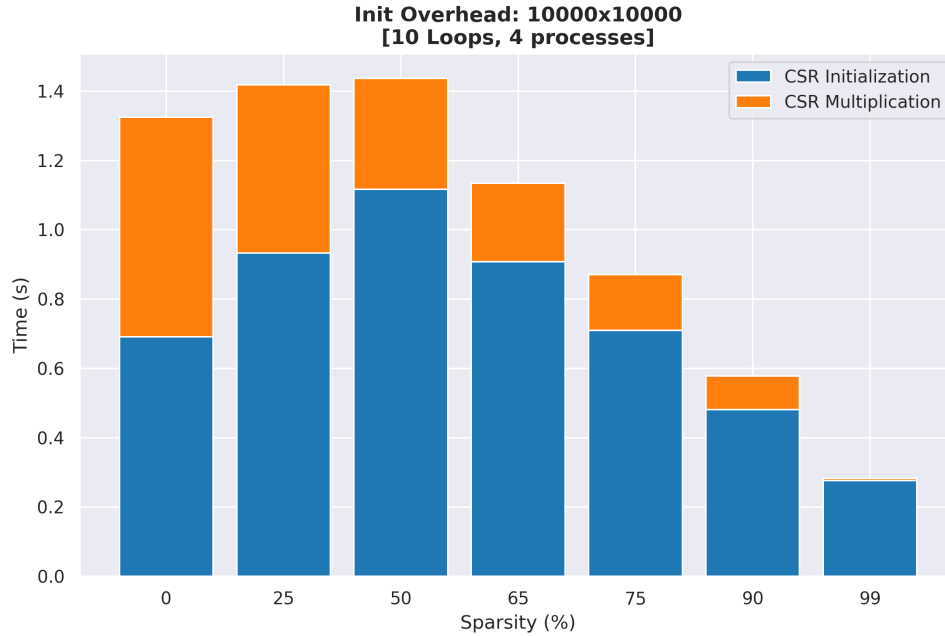
(β') Total execution time for Dense format across sparsity levels.

Σχήμα 10: Detailed computational performance analysis: (a) highlights the sensitivity of CSR to non-zero density, whereas (b) illustrates the overhead of the dense algorithm in sparse contexts.

Ο χρόνος παραμένει σταθερός για την πυκνή αναπαράσταση, ενώ για τη μορφή CSR ο χρόνος μειώνεται όσο αυξάνεται η αραιότητα των πινάκων.

Το κόστος αρχικοποίησης των τριών πινάκων της αναπαράστασης CSR είναι ιδιαίτερα σημαντικό όταν ο αριθμός των επαναλήψεων είναι περιορισμένος, αλλά το σχετικό overhead αποσβένεται καθώς οι επαναλήψεις

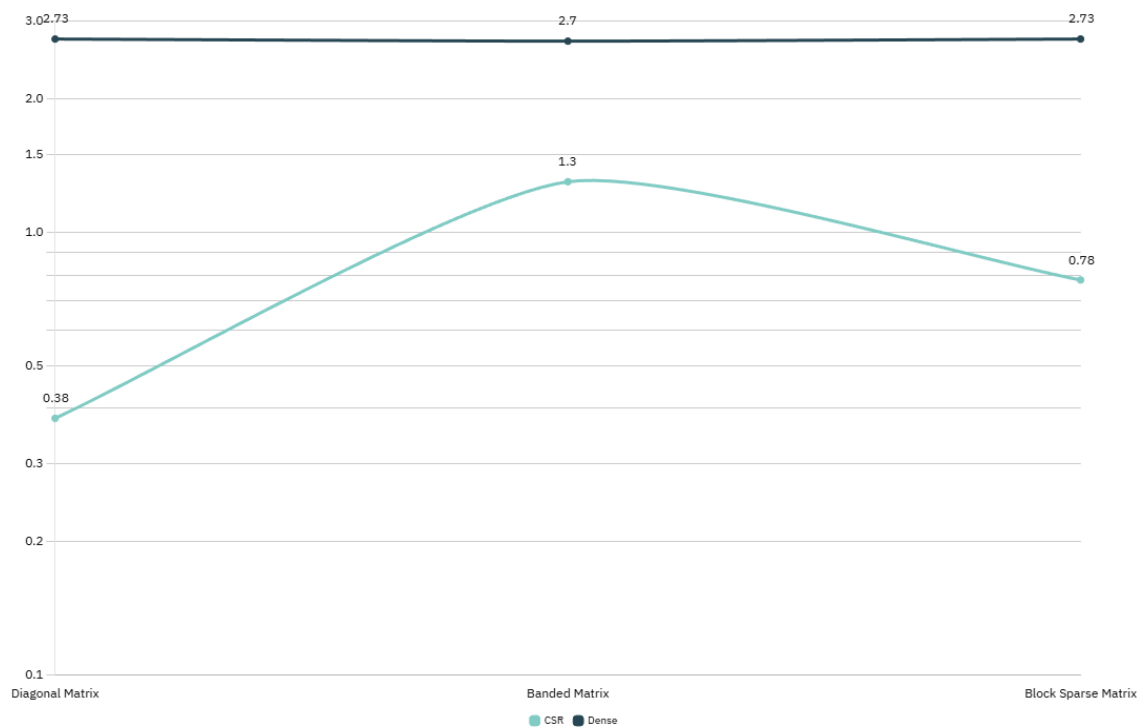
αυξάνονται. Γενικότερα, όσο μεγαλύτερος είναι ο όγκος των υπολογισμών που αποφεύγουμε, τόσο περισσότερο δικαιολογείται το υπολογιστικό κόστος της μετατροπής από την `dense` μορφή. Η προσέγγιση αυτή κρίνεται ιδανική για επαναληπτικές μεθόδους, όπως τα προβλήματα σταθερού σημείου.



(α') Initialization overhead for 20 iterations (5000×5000 matrix).

Σχήμα 11: Analysis of CSR initialization overhead: The figures illustrate how the cost of building the CSR structure is amortized over an increasing number of multiplication iterations (10 vs. 20).

Τέλος, εξετάστηκαν ορισμένες ειδικές περιπτώσεις πινάκων που εμφανίζονται συχνά σε πραγματικές εφαρμογές, ώστε να αξιολογηθεί η επίδοση της αναπαράστασης CSR σε ρεαλιστικά σενάρια. Σημειώνεται ότι οι χρόνοι που παρουσιάζονται για τη μορφή CSR συμπεριλαμβάνουν και το κόστος της αρχικοποίησης:



Σχήμα 12: Special cases

Όπως παρατηρείται, επιτυγχάνεται εξαιρετικά υψηλή επιτάχυνση (speedup), ιδιαίτερα στην περίπτωση των διαγώνιων πινάκων. Αυτό εξηγείται από το γεγονός ότι η μορφή CSR μειώνει την υπολογιστική πολυπλοκότητα στο $O(n)$, εξαλείφοντας πλήρως την ανάγκη επεξεργασίας του τεράστιου όγκου μηδενικών στοιχείων που χαρακτηρίζει αυτούς τους πίνακες.

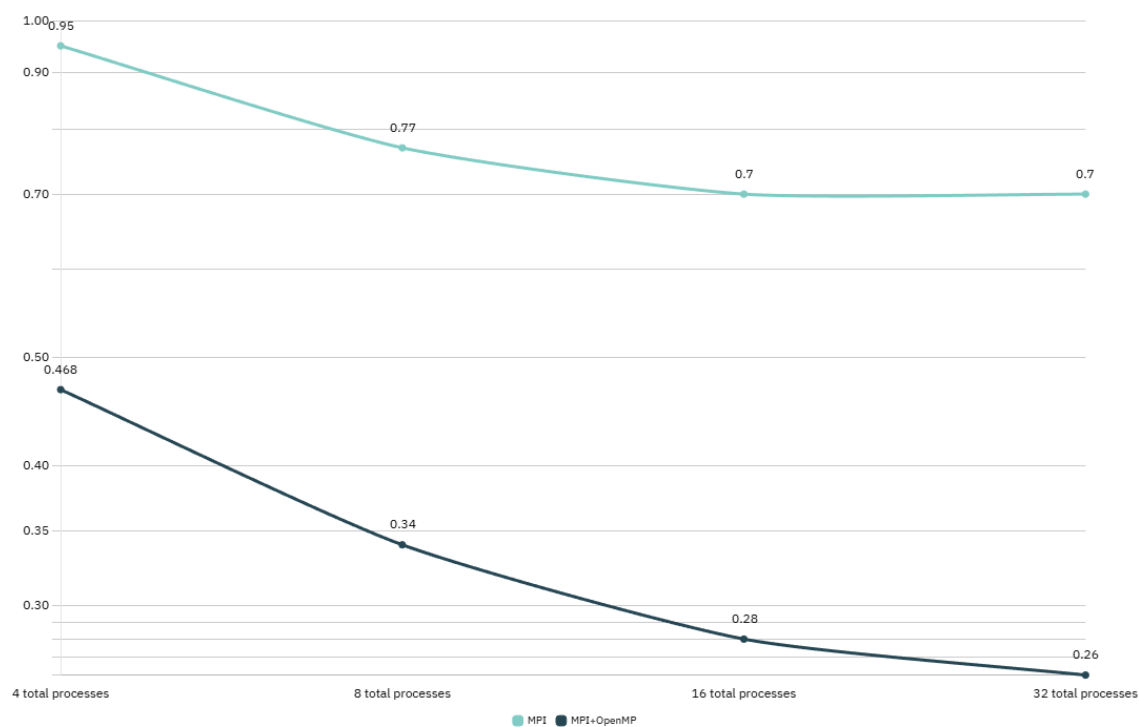
3 Ασκήση 3

3.1 Περιγραφή Προβληματος και λυσης

Στο επόμενο στάδιο της μελέτης, πραγματοποιήθηκε ο συνδυασμός των προτύπων MPI και OpenMP για τη δημιουργία μιας υβριδικής υλοποίησης, η οποία συγκρίθηκε με την αμιγή (*pure*) υλοποίηση MPI. Η υβριδική προσέγγιση αναπτύχθηκε μέσω της ενσωμάτωσης των παραλληλοποιημένων τμημάτων κώδικα του OpenMP στον κορμό της υποδομής του MPI.

3.2 Πειραματικά αποτελεσματα και σχολιασμός

Στο πλαίσιο των πειραματικών μετρήσεων, πραγματοποιήθηκε σύγκριση της υβριδικής υλοποίησης έναντι της αμιγούς MPI, διατηρώντας σταθερό τον συνολικό αριθμό υπολογιστικών πόρων (διεργασιών και νημάτων). Παρακάτω παρατίθενται τα διαγράμματα επιδόσεων των δύο προγραμμάτων για διαφορετικούς συνδυασμούς κόμβων και νημάτων υπό σταθερές παραμέτρους προβλήματος. Σημειώνεται ότι οι χρόνοι εκτέλεσης δεν συμπεριλαμβάνουν την καθυστέρηση της διανομής των δεδομένων (*scattering*), καθώς δεν παρατηρήθηκε αξιοσημείωτη απόκλιση μεταξύ των δύο υλοποιήσεων στο συγκεκριμένο στάδιο.



Σχήμα 13: Performance scaling comparison between pure MPI and hybrid MPI+OpenMP implementations for a fixed number of total execution units.

Όπως προκύπτει από τα αποτελέσματα, η υβριδική προσέγγιση υπερτερεί σημαντικά σε ταχύτητα εκτέλεσης, επιδεικνύοντας παράλληλα βελτιωμένη κλιμακωσιμότητα (*scalability*). Η σημαντικότερη βελτίωση εντοπίζεται στην παράλληλη αρχικοποίηση των πινάκων, η οποία υλοποιείται αποτελεσματικότερα μέσω του *OpenMP*, αποφεύγοντας το αυξημένο *overhead* που θα απαιτούσε μια αντίστοιχη υλοποίηση σε *MPI*. Η υβριδική λύση συνδυάζει τα πλεονεκτήματα και των δύο μοντέλων: την ικανότητα κλιμάκωσης σε πολλούς κόμβους και τον αποδοτικό, χαμηλού κόστους (*lightweight*) παραλληλισμό εντός του κόμβου. Τα πειραματικά δεδομένα επιβεβαιώνουν ότι η υβριδική υλοποίηση αποτελεί τη βέλτιστη λύση.